Concurrencia sobre STM32: del super-loop bloqueante a las interrupciones

Laboratorio 2 — Sistemas Electrónicos de Instrumentación

Pedro Villar | 24 de octubre de 2026

Los seis desafíos se resolvieron sobre una **Nucleo-F411RE** en lugar de la F411E-DISCO de la guía, así que hubo que adaptar pines: el LED verde es el de la placa (PA5), el naranja, rojo y azul se cablearon externos (PA6, PA7, PB6), y el botón es el de la placa (PC13), que es *activo en bajo* — al revés que en la guía, por eso en el código "presionado" se compara contra `GPIO_PIN_RESET`. Todo corre con el HSI de 16 MHz sin PLL: con prescaler 16000 los timers cuentan a 1 kHz y el período en ms se escribe directo en el ARR. Acá va solo el lazo principal de cada desafío; el proyecto completo (CubeMX + CMake, un archivo por desafío) está en el repositorio.

Desafío 1 — Memoria de estado y antirrebote bloqueante

```
while (1) {
  if (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET) {    // presionado
    HAL_Delay(50);                                          // antirrebote bloqueante
    if (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET) {  // era real
      HAL_GPIO_TogglePin(LED_ROJO_GPIO_Port, LED_ROJO_Pin);
      while (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET);
    }
  }
}
```

La clave es detectar el *flanco* y no el nivel: sin el `while` final, mantener apretado dispararía toggles sin parar. El rebote se filtra esperando 50 ms y releyendo. Funciona perfecto… porque es la única tarea: entre el delay y la espera a que se suelte el botón, el procesador queda clavado sin poder atender nada más. Ese es exactamente el problema que el resto del laboratorio va a ir resolviendo.

Desafío 2 — Temporización cooperativa con `HAL_GetTick()`

```
while (1) {
  uint32_t millisActual = HAL_GetTick();
  if (millisActual - millisAnterior >= 250) {            // ¿le toca al LED azul?
    millisAnterior = millisActual;
    HAL_GPIO_TogglePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin);
  }
  if (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET) // se lee siempre
    HAL_GPIO_WritePin(LED_VERDE_GPIO_Port, LED_VERDE_Pin, GPIO_PIN_SET);
  else
    HAL_GPIO_WritePin(LED_VERDE_GPIO_Port, LED_VERDE_Pin, GPIO_PIN_RESET);
}
```

En vez de frenar 250 ms, el lazo pregunta en cada vuelta "¿ya pasaron?". El loop gira miles de veces por segundo, así que el verde responde al botón al instante mientras el azul mantiene su ritmo. Detalle fino: la resta entre `uint32_t` sigue dando bien incluso cuando el contador de ticks desborda. Esto es concurrencia *cooperativa*: las tareas conviven porque ninguna bloquea.

Desafío 3 — Máquina de estados

```
while (1) {
  GPIO_PinState botonActual = HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin);
  if (botonActual == GPIO_PIN_RESET && botonAnterior == GPIO_PIN_SET) { // flanco
    HAL_Delay(20);                        // antirrebote corto, permitido en la transición
    estado = (estado + 1) % 3;            // 0 apagado -> 1 lento -> 2 rápido -> 0 ...
    ApagarTodosLosLeds();
  }
  botonAnterior = botonActual;

  if (estado == 1) periodo = 500;         // 1 Hz
  if (estado == 2) periodo = 100;         // 5 Hz
  if (estado != 0 && HAL_GetTick() - millisAnterior >= periodo) {
    millisAnterior = HAL_GetTick();
    InvertirTodosLosLeds();               // toggle de los cuatro LEDs
  }
}
```

Acá se integran las dos técnicas anteriores. El flanco se detecta comparando la lectura actual

con la anterior (el lazo ya no puede darse el lujo de quedarse esperando), y la variable `estado` es la memoria del sistema: define el período de parpadeo sin tocar la lógica del botón. Por eso la respuesta a la presión es inmediata sin importar qué tan rápido parpadeen los LEDs.

Desafío 4 — Timer de hardware por polling

```
HAL_TIM_Base_Start(&htim2);              // desborda cada 100 ms (PSC 16000-1)
while (1) {
  if (__HAL_TIM_GET_FLAG(&htim2, TIM_FLAG_UPDATE) != RESET) {
    __HAL_TIM_CLEAR_FLAG(&htim2, TIM_FLAG_UPDATE);
    AvanzarSecuenciaLEDs();              // V-N-R-A-R-N, un paso por desborde
  }
}
```

El procesador deja de contar tiempo: lo cuenta un periférico dedicado, con precisión de hardware e independiente del SysTick. Pero todavía hay que *preguntar*: si el lazo estuviera ocupado en otra cosa, el flag quedaría levantado esperando y la secuencia se atrasaría. La ventaja sobre el desafío 2 es la precisión; la limitación sigue siendo el polling.

Desafío 5 — Interrupciones

```
HAL_TIM_Base_Start_IT(&htim3);           // TIM3 interrumpe cada 500 ms
while (1) {
  HAL_GPIO_TogglePin(LED_VERDE_GPIO_Port, LED_VERDE_Pin); // se nota lo lento del main
  HAL_Delay(5000);                       // main bloqueado 5 segundos
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  if (htim->Instance == TIM3)
    HAL_GPIO_TogglePin(LED_NARANJA_GPIO_Port, LED_NARANJA_Pin);
}
```

La prueba de fuego: el main está preso de un `HAL_Delay(5000)` (el verde cambia cada 5 segundos y lo delata), y sin embargo el naranja parpadea a 1 Hz exacto. El NVIC frena al main donde esté, ejecuta el callback y le devuelve el control como si nada. Nadie pregunta por ningún flag: el hardware *avisa*. Esto ya es concurrencia preemptiva.

Desafío 6 — Antirrebote por muestreo en ISR

```
volatile uint8_t bandera_boton = 0;      // la escribe la interrupción

while (1) {                              // el main solo mira la bandera
  if (bandera_boton) {
    bandera_boton = 0;
    HAL_GPIO_TogglePin(LED_AZUL_GPIO_Port, LED_AZUL_Pin);
  }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  if (htim->Instance == TIM4) {          // cada 10 ms
    static uint8_t contador = 0;
    if (HAL_GPIO_ReadPin(BOTON_GPIO_Port, BOTON_Pin) == GPIO_PIN_RESET) {
      contador++;
      if (contador == 5) bandera_boton = 1;   // 5 lecturas x 10 ms = 50 ms estable
    } else {
      contador = 0;                      // ruido o botón suelto: arrancar de nuevo
    }
  }
}
```

El mismo problema del desafío 1, resuelto sin bloquear ni un microsegundo: el TIM4 muestrea el botón cada 10 ms y recién con 5 lecturas seguidas en "presionado" da el flanco por válido — el ruido nunca junta 5. ISR y main se comunican por una bandera compartida, y ahí `volatile` no es decorativo: sin él, el compilador ve que el main nunca escribe la variable, asume que vale siempre cero y el toggle no ocurre jamás.

Conclusiones

El laboratorio es, en el fondo, una sola pregunta repetida seis veces: ¿quién es el dueño del tiempo? Primero lo tiene el procesador y lo gasta esperando (`HAL_Delay`); después lo administra el propio lazo preguntando la hora (`HAL_GetTick`); después lo cuenta un periférico y el procesador solo consulta (polling del flag); y al final el periférico directamente interrumpe y avisa. Cada paso libera más al main, a cambio de un modelo mental más complejo: con interrupciones el programa deja de ser una secuencia que uno puede seguir con el dedo, y aparecen problemas nuevos que en software "normal" no existen, como que el compilador optimice de más una variable compartida si uno se olvida del `volatile`. Me quedo con una regla práctica que resume todo: *bloquear solo se puede cuando no hay nada más que atender* — y casi siempre hay algo más que atender. Viniendo del laboratorio 1, donde la concurrencia era literal porque cada bloque era hardware funcionando en paralelo, acá se ve el mismo objetivo pero desde un solo hilo de ejecución: la FPGA es paralela por naturaleza y hay que esforzarse para secuenciar; el micro es secuencial por naturaleza y hay que esforzarse para que parezca paralelo.